

STDM: A Shared Tangent-Plane FP32 Divider–Multiplier with Multi-Level Accuracy–PPA Tradeoffs

Pingchuan Dong

Abstract—Approximate floating-point division and multiplication are usually designed as independent operators, even when a processing element issues only one of them at a time. This paper presents STDM, four FP32 divider–multiplier designs that share a piecewise tangent-plane mantissa datapath. We prove that local approximations of multiplication and division use the same two residual terms and a cell-dependent constant. Reciprocal-square scaling for division is folded into these coefficients, eliminating a separate post-plane multiplier. We also introduce an exact unsigned recoding of centered residuals: the operation sign is absorbed into the selected constant and a bitwise complement of the short divisor residual. The resulting datapath uses two unsigned products and one positive accumulation without changing any approximate output. In a TSMC 65-nm implementation, sharing at L1–L3 reduces area by 17.2–19.0%, power by 31.6–42.4%, and area–delay product by 11.8–14.4% relative to matched separate cores, with at most 6.5% delay overhead. DIV-only comparisons include PACE, FPD2D, PLSAD, and other approximate FP32 dividers under a common local flow. At L3, the complete shared unit reduces area, delay, power, and area–delay product by 92.5%, 74.5%, 98.1%, and 98.1%, respectively, relative to an exact DesignWare FP32 DIV+MUL unit.

Index Terms—approximate computing, floating-point arithmetic, divider, multiplier, tangent-plane approximation, arithmetic sharing, ASIC

1 Introduction

Approximate arithmetic reduces the cost of floating-point computation when an application can tolerate bounded numerical error. Most approximate dividers and multipliers are nevertheless optimized as independent units. A processing element that issues one operation at a time then pays for two mantissa datapaths, even though both operate on the same normalized significands and share normalization and packing. Avoiding this duplication requires more than placing two compact operators behind a selector: their approximations must be expressed in a common arithmetic form.

Prior dividers simplify digit recurrence, reciprocal evaluation, logarithmic conversion, or a fitted quotient surface. PACE progressively corrects a piecewise reciprocal [2]; PLSAD and FPD2D show that quantized local planes can give particularly low-cost DIV-only implementations [17], [18]. OAM uses midpoint tangent planes for multiplication [1]. These results establish strong operator-level tradeoffs, but leave open whether a planar FP32 divider and multiplier can share their dominant arithmetic without reducing them to a logarithmic approximation.

STDM answers this question with four fixed-level FP32 divider–multiplier designs. For a selected cell, let k_x, k_y be its midpoint and let $r_x = x - k_x$, $r_y = y - k_y$. The multiplier plane is $k_x k_y + k_y r_x + k_x r_y$, while the divider plane is $(k_x k_y + k_y r_x - k_x r_y)/k_y^2$. Hence both operations are affine functions of the same two residuals. STDM folds the reciprocal-square factor into the divider slopes and constant, so either mode is evaluated as one table-selected constant plus two products. This avoids both an inter-level correction chain and a post-plane coefficient multiplier.

The remaining problem is realizing the signed affine form efficiently. STDM truncates only the low residual bits selected for each level and then applies an exact unsigned recoding. Adding the residual half-range converts each short signed residual to an unsigned field. In DIV mode, a bitwise complement of the y field and a precomputed constant adjustment implement the negative term. MUL and DIV therefore use the same two unsigned products and one positive accumulation. This transformation changes circuit organization but not the computed approximation. Reciprocal coefficients are calibrated using the complete packed FP32 result before being folded into the slopes and constant.

The contributions are as follows:

- We derive the algebraic relation between midpoint tangent planes for FP32 division and multiplication and use it to share partition selection, two residual products, accumulation, normalization, and packing.
- We fold reciprocal-square scaling into cell-specific divider slopes and constants, removing the separate coefficient multiplier, and introduce an exact unsigned residual recoding that removes signed products and the wide DIV subtraction from the shared core.
- We analyze partition, residual truncation, and coefficient quantization error, and calibrate coefficients for the complete FP32 datapath rather than for the reciprocal alone.
- We compare matched shared and separate implementations with identical outputs. At L1–L3, sharing reduces area by 17.2–19.0%, power by 31.6–42.4%, and area–delay product by 11.8–14.4%, with at most 6.5% delay overhead. DIV-only evaluation includes PACE, PLSAD, FPD2D, and other prior architectures under the same local synthesis flow.

2 Related Work

2.1 Approximate FP Division

Approximate dividers can be grouped by the arithmetic they simplify. Truncation and approximate digit-recurrence designs reduce work in a restoring, non-restoring, or subtractor-array-style core. Chen et al. simplify a non-restoring divider, Hashemi et al. introduce a dynamic approximate divider, and TruncApp truncates divider computation [7], [8], [13]. They retain a familiar quotient-generation structure, although an array implementation can still have substantial logic depth or require a reduced exact core. Logarithmic designs use a leading-one detector and an approximation to $\log_2(1+f)$ so that division becomes subtraction in the log domain. LEAD and FaNZeD are representative examples [14], [12]. CADE similarly replaces FP mantissa division with subtraction and uses a table for error correction [9].

Reciprocal-based dividers approximate $1/y$ before combining it with the dividend. SAADI-EC incrementally refines a reciprocal through Taylor-series terms, while ILAFD combines an approximate reciprocal with an iterative logarithmic multiplier [10], [11]. QIAD uses quadratic interpolation, whereas PACE uses a piecewise reciprocal formulation formed from shifts, additions, and level-wise error compensation [15], [2]. The approach can be accurate, but a conventional reciprocal datapath often needs one or more multipliers. Surface-fitting designs approximate the mantissa surface x/y directly with local planes. Wu et al. introduced an earlier curve-fitting divider. PLSAD combines piecewise linear fitting with coefficient realization and an approximate lower-part adder, while FPD2D optimizes quantized coefficients over a two-dimensional tiling of both mantissas [16], [17], [18]. Their shift-add and carry-save realizations demonstrate the efficiency possible when a DIV-only surface fit is aggressively specialized. The categories overlap in individual designs, but they expose an important limitation for this work: a DIV-only approximation does not by itself yield a low-cost multiplier that shares its arithmetic.

Among the works reviewed here, SIMDive is the closest approximate combined operator. It shares Mitchell log-domain arithmetic between multiplication and division [3]. Its published design, however, operates on 8–32-bit integers in SIMD FPGA lanes. STDM is a scalar FP32 ASIC design whose measured cost includes sign, exponent, normalization, and packing. An FP32 wrapper around SIMDive would be a new adaptation that changes both its error and hardware cost. We therefore discuss SIMDive as related work but do not use it as a PPA baseline.

Exact FP units have also combined multiplication and division to save hardware, but they solve a different problem. Chen et al. share signed-digit adders, a termination multiplier, and normalization stages in a single-precision fused unit [4]; unlike STDM, its target is an exact floating-point result rather than a selectable approximation level.

The DIVA FPU reuses a pipelined mantissa multiplier across Taylor-series divider iterations [5], so its cost is tied to a sequential schedule rather than a one-pass combinational approximation. GRFPU similarly reuses multiplication hardware for iterative division and square root and supports both IEEE-754 single and double precision [6]. Its formats, operation set, and multicycle interface differ from STDM. These differences prevent a controlled PPA comparison, so the exact designs are used only to place arithmetic sharing in context.

STDM builds on a different observation. The locally optimal divider plane derived in this work and the OAM multiplier plane have a common midpoint-partition structure and related level-wise corrections. Unlike the DIV-only architectures above, it makes that algebraic relation an architectural objective: one shared datapath implements either operation without duplicating partition, residual, and plane logic.

3 Derivation and Shared Architecture

Compared with integer arithmetic, floating-point arithmetic requires separate handling of the sign, significand, and exponent. Following IEEE 754 [19], a normalized binary floating-point operand can be written as

$$F_x = (-1)^{s_x} x 2^{E_x}, \quad x \in [1, 2),$$

where $sign_x, x, E_x$ refer to the sign, mantissa and exponent of F_x respectively. Thus for two FP numbers F_x and F_y , the division is:

$$\frac{F_x}{F_y} = sign_x sign_y \times \frac{x}{y} \times 2^{E_x - E_y}.$$

In a general processor, for an FP division, the sign bits are computed by an XOR operation, and the exponent bits are computed by a subtractor. The division $\frac{x}{y}$ of two mantissas is much more costly than the computations of signs and exponents. The motivation of this work is to propose an energy efficient divider for $\frac{x}{y}$ between mantissas.

3.1 Preliminaries: OAM

OAM approximates the mantissa product in FP multiplication after the same front-end decomposition. The exact mantissa product is $h(x, y) = xy$. OAM divides the mantissa domain into local rectangular regions and replaces $h(x, y)$ in each region by a tangent plane. For a generic cell $[x_1, x_2] \times [y_1, y_2]$, let the tangent point be (k_x, k_y) . The first-order expansion of xy at this point gives

$$\begin{aligned} \hat{m} &= k_x k_y + k_y(x - k_x) + k_x(y - k_y) \\ &= -k_x k_y + k_y x + k_x y. \end{aligned} \tag{1}$$

The OAM formulation chooses the tangent point by minimizing the integral error over the cell. With

$$E_M(k_x, k_y) = \int_{x_1}^{x_2} \int_{y_1}^{y_2} [xy - (-k_x k_y + k_y x + k_x y)] dy dx, \tag{2}$$

the stationarity conditions yield

$$k_x = \frac{x_1 + x_2}{2}, \quad k_y = \frac{y_1 + y_2}{2}. \quad (3)$$

Thus the selected tangent point is simply the midpoint of the local mantissa cell. At level n , OAM partitions each mantissa axis into 2^n intervals. If i_x^n and i_y^n are the selected interval indices, the corresponding midpoints are

$$k_x^n = 1 + \left(i_x^n + \frac{1}{2}\right) 2^{-n}, \quad k_y^n = 1 + \left(i_y^n + \frac{1}{2}\right) 2^{-n}. \quad (4)$$

The level therefore controls how fine the local plane is: L0 uses one plane over the whole normalized domain, while L1–L3 progressively refine the partition. STDMM starts from this OAM structure and asks whether the divider’s mantissa quotient can be expressed with the same midpoint partition and the same elementary plane terms.

3.2 Divider Tangent-Plane Equation Derivation

3.2.1 Direct Derivation

For any local cell $[x_1, x_2] \times [y_1, y_2] \subset [1, 2] \times [1, 2]$, we first approximate the divider surface $z = x/y$ directly. The tangent plane at $(x_0, y_0, z_0 = x_0/y_0)$ is

$$z = \frac{x_0}{y_0} + \frac{1}{y_0}(x - x_0) - \frac{x_0}{y_0^2}(y - y_0) = \frac{1}{y_0}x - \frac{x_0}{y_0^2}y + \frac{x_0}{y_0}. \quad (5)$$

The tangent point is chosen by minimizing the integral error over the cell:

$$E_D(x_0, y_0) = \int_{x_1}^{x_2} \int_{y_1}^{y_2} \left[\frac{x}{y} - \left(\frac{x_0}{y_0} + \frac{1}{y_0}x - \frac{x_0}{y_0^2}y \right) \right] dy dx. \quad (6)$$

Let $A = x_2 - x_1$, $B = y_2 - y_1$, $S_x = (x_2^2 - x_1^2)/2$, and $S_y = (y_2^2 - y_1^2)/2$. The stationarity conditions are

$$\frac{\partial E_D}{\partial x_0} = -\frac{AB}{y_0} + \frac{AS_y}{y_0^2} = 0, \quad (7)$$

and

$$\frac{\partial E_D}{\partial y_0} = \frac{x_0 AB}{y_0^2} + \frac{S_x B}{y_0^2} - \frac{2x_0 AS_y}{y_0^3} = 0. \quad (8)$$

They give

$$y_0 = \frac{S_y}{B} = \frac{y_1 + y_2}{2}, \quad x_0 = \frac{S_x}{A} = \frac{x_1 + x_2}{2}. \quad (9)$$

Thus the locally optimized tangent point is the cell midpoint. With $k_x = (x_1 + x_2)/2$ and $k_y = (y_1 + y_2)/2$, the divider approximation is

$$\hat{d} = \frac{k_x}{k_y} + \frac{1}{k_y}(x - k_x) - \frac{k_x}{k_y^2}(y - k_y) = \frac{k_x k_y + k_y x - k_x y}{k_y^2}. \quad (10)$$

3.2.2 OAM-Based Equivalent Derivation

The same divider plane can also be obtained from a shorter construction that uses the OAM product plane. On $[y_1, y_2]$, approximate $1/y$ by the tangent line at p ,

$$g(y) = -\frac{y}{p^2} + \frac{2}{p}. \quad (11)$$

Choosing p to minimize the integral error gives

$$0 = E'_R(p) = (y_1^2 - y_2^2)p^{-3} + 2(y_2 - y_1)p^{-2}, \quad (12)$$

and therefore $p = (y_1 + y_2)/2 = k_y$. Combining this reciprocal tangent with the OAM product approximation in (1) gives

$$\begin{aligned} \frac{x}{y} &= x \cdot \frac{1}{y} \approx -\frac{1}{k_y^2}xy + \frac{2}{k_y}x \\ &\approx -\frac{1}{k_y^2}(-k_x k_y + k_y x + k_x y) + \frac{2}{k_y}x \\ &= \frac{1}{k_y^2}(k_x k_y + k_y x - k_x y). \end{aligned} \quad (13)$$

Equations (10) and (13) are algebraically identical. The former is the first-order Taylor approximation of x/y about the cell midpoint (k_x, k_y) , whereas the latter first applies the tangent approximation of $1/y$ at k_y and then substitutes the first-order OAM approximation of xy about the same midpoint. Because both constructions retain the same first-order terms at the same expansion point, they produce the same affine divider plane. This equivalence allows the divider to reuse the midpoint partition and product-plane terms of OAM.

3.3 Architecture for Multi-Level Approximate Divider

The two derivations above give a level-independent local form for division. For the level- n rectangular cell containing (x, y) , let k_x^n and k_y^n denote the midpoint coordinates. The proposed divider approximates the mantissa quotient as

$$\hat{d}_n = \frac{w_n}{(k_y^n)^2}, \quad w_n = k_x^n k_y^n + k_y^n x - k_x^n y. \quad (14)$$

For the same cell and midpoint, OAM approximates the mantissa product as

$$\hat{m}_n = z_M^n = -k_x^n k_y^n + k_y^n x + k_x^n y. \quad (15)$$

Thus the divider numerator and multiplier plane use the same partition, midpoint coordinates, and primitive terms, while division changes the sign of the $k_x^n y$ contribution and scales the numerator by the reciprocal-square constant $1/(k_y^n)^2$.

Fig. 1 summarizes how this fixed-level formulation extends to the complete FP32 divider-multiplier. The operation modes share the partition and residual arithmetic. The reciprocal-square factor is absorbed into the divider’s cell coefficients rather than applied by a later multiplier.

For each implementation, the target level selects one local plane directly at elaboration rather than retaining a chain of corrections from L0 to that level. This distinction preserves the algebraic sharing between DIV and MUL without requiring inter-level accumulation of an incremental $\Delta z_i/\Delta w_i$ architecture.

The initial FP32 mantissa domain is $[1, 2] \times [1, 2]$. Level 0 uses this whole domain as one cell. At level 1, each



Fig. 1. Shared selected-level organization of the proposed STDM FP32 divider-multiplier. DIV and MUL select mode-specific coefficients for a common two-product affine datapath.

axis is split at 1.5, producing four rectangular subregions. In general, level n partitions each axis into 2^n equal intervals:

$$[x_{p-1}^n, x_p^n) \times [y_{q-1}^n, y_q^n), \quad (16)$$

where $x_p^n = 1 + p/2^n$ and $y_q^n = 1 + q/2^n$ define the interval boundaries along the x - and y -axes, respectively. Fig. 2 visualizes this recursive subdivision for L0–L2; L3 continues the same bisection once more along each axis.

The midpoint of the selected cell is therefore

$$\begin{aligned} k_x^n &= 1 + \frac{2p(n) - 1}{2^{n+1}}, \\ k_y^n &= 1 + \frac{2q(n) - 1}{2^{n+1}}. \end{aligned} \quad (17)$$

where $p(n)$ and $q(n)$ are determined by the leading mantissa bits of x and y . Equivalently, starting from $k_x^0 = k_y^0 = 3/2$, each deeper level updates the midpoint by one signed binary step:

$$\begin{aligned} k_x^n &= k_x^{n-1} + \sigma_x(n)2^{-(n+1)}, \\ k_y^n &= k_y^{n-1} + \sigma_y(n)2^{-(n+1)}. \end{aligned} \quad (18)$$

where $\sigma_x(n), \sigma_y(n) \in \{-1, +1\}$ are selected by the n th fractional bits.

These midpoint coordinates introduce no representation error. For L0–L3, every midpoint is a dyadic rational with denominator at most 16 and is stored exactly as the integer code $K = 16k$. Thus midpoint selection and the products used to form the untruncated local plane do not quantize k_x^n or k_y^n .

Fig. 3 shows that the same operand pair (x, y) lies in one cell at each partition level. As the level increases, the grid becomes finer; the leading mantissa bits identify the finer cell that contains (x, y) , and the center of that cell defines the level-dependent midpoint (k_x^n, k_y^n) used to evaluate the selected tangent plane.

The remaining divider-specific factor is the reciprocal-square scale. Since k_y^n is fixed once the level and y -interval are known, the scale can be precomputed and selected from a small table. For the L0–L3 levels used in this work,

the ideal constants are

$$\begin{aligned} \text{L0: } & \frac{1}{(3/2)^2}, \\ \text{L1: } & \begin{cases} \frac{1}{(5/4)^2}, & y[1] = 0, \\ \frac{1}{(7/4)^2}, & y[1] = 1, \end{cases} \\ \text{L2: } & \begin{cases} \frac{1}{(9/8)^2}, & y[1:2] = 00, \\ \frac{1}{(11/8)^2}, & y[1:2] = 01, \\ \frac{1}{(13/8)^2}, & y[1:2] = 10, \\ \frac{1}{(15/8)^2}, & y[1:2] = 11, \end{cases} \\ \text{L3: } & \begin{cases} \frac{1}{(17/16)^2}, & y[1:3] = 000, \\ \frac{1}{(19/16)^2}, & y[1:3] = 001, \\ \frac{1}{(21/16)^2}, & y[1:3] = 010, \\ \frac{1}{(23/16)^2}, & y[1:3] = 011, \\ \frac{1}{(25/16)^2}, & y[1:3] = 100, \\ \frac{1}{(27/16)^2}, & y[1:3] = 101, \\ \frac{1}{(29/16)^2}, & y[1:3] = 110, \\ \frac{1}{(31/16)^2}, & y[1:3] = 111. \end{cases} \end{aligned} \quad (19)$$

Unlike the midpoints themselves, these reciprocal-square values generally do not have finite binary representations. In particular, if $k_y^n = K_y^n/16$, then $1/(k_y^n)^2 = 256/(K_y^n)^2$; after reduction, the denominator normally retains an odd factor. For example, L0 requires $1/(3/2)^2 = 4/9$, whose binary expansion repeats. As discussed in the next section, STDM first quantizes and calibrates these values and then absorbs them into the divider slopes and constant. The reciprocal code is therefore a design parameter used to generate the table, not a separate multiplier operand in the final RTL. This approximation does not alter the exact midpoint codes.

4 Circuit Implementation and Optimization

The OAM derivation can be implemented recursively by evaluating L0 and adding one correction for every deeper level. STDM instead fixes the target level at elaboration, decodes the leading mantissa bits into the selected cell, and evaluates that cell's affine function once. Before precision reduction, direct selection is algebraically equivalent to

$$P_n = P_0 + \sum_{i=1}^n \Delta P_i, \quad (20)$$

but it avoids the intermediate midpoint updates, correction generators, and accumulation tree when only level n is observable.

4.1 Centered Residuals and Coefficient Absorption

For the selected cell, define

$$r_x^n = x - k_x^n, \quad r_y^n = y - k_y^n. \quad (21)$$

Substitution into the two planes gives

$$\hat{m}_n = k_x^n k_y^n + k_y^n r_x^n + k_x^n r_y^n, \quad (22)$$

$$\hat{d}_n = c_n (k_x^n k_y^n + k_y^n r_x^n - k_x^n r_y^n), \quad c_n = \frac{1}{(k_y^n)^2}. \quad (23)$$

Centering is exact and bounds each residual by half the cell width. The leading n fraction bits select k_x^n and k_y^n ; the midpoint coordinates remain the exact dyadic values derived in Section III.

Let R_x^n and R_y^n be the Q23 integer residuals. After dropping D_r low bits, the retained signed integers are

$$\rho_x = \left\lfloor \frac{R_x^n}{2^{D_r}} \right\rfloor, \quad \rho_y = \left\lfloor \frac{R_y^n}{2^{D_r}} \right\rfloor. \quad (24)$$

The residual width is only $b = 23 - n - D_r$ bits. Rather than first constructing the unscaled divider plane and then multiplying it by c_n , STDm folds the reciprocal factor into the cell coefficients. With S fractional bits for the slopes and $F = S + 23 - D_r$ fractional bits for the constant, the signed forms are

$$Q_M = T_M + A_M \rho_x + B_M \rho_y, \quad (25)$$

$$Q_D = T_D + A_D \rho_x - B_D \rho_y, \quad (26)$$

where

$$\begin{aligned} A_M &= \text{round}(k_x^n 2^S), & B_M &= \text{round}(k_y^n 2^S), \\ A_D &= \text{round}(\bar{c}_n k_x^n 2^S), & B_D &= \text{round}(\bar{c}_n k_y^n 2^S), \end{aligned} \quad (27)$$

$$T_M = \text{round}((k_x^n k_y^n + b_M) 2^F), \quad T_D = \text{round}(\bar{c}_n k_x^n k_y^n 2^F).$$

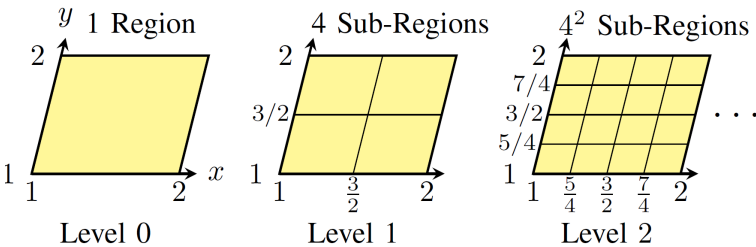


Fig. 2. Multi-level partition of the normalized mantissa domain. Each level bisects every interval along both axes.

Here, b_M is the residual-truncation bias derived below, and $\bar{c}_n$ is the calibrated reciprocal-square coefficient. The represented mantissa value is $Q2^{-F}$; equivalently, the RTL left shifts the integer sum by $D_r - S$ to restore Q23 alignment. Equation (27) is evaluated when the tables are generated. Consequently, no reciprocal multiplication remains in the synthesized datapath.

For multiplication, truncating a Q23 residual has step $\Delta_r = 2^{D_r-23}$. If the discarded portions are approximately uniform in a cell, each has mean $\Delta_r/2$, giving the mantissa-domain correction

$$b_M = (k_x^n + k_y^n) 2^{D_r-24}. \quad (28)$$

This correction is included in T_M and therefore adds no hardware.

4.2 Exact Unsigned Residual Recoding

Equations (25) and (26) still imply signed products and, for DIV, a post-product subtraction. STDm removes both without changing the integer result. Let $H = 2^{b-1}$ and define the unsigned b -bit fields

$$u = \rho_x + H, \quad v = \rho_y + H. \quad (29)$$

Because the midpoint is the center of a binary interval, u and v are exactly the retained local mantissa fields; no subtractor is required. For MUL,

$$Q_M = \underbrace{T_M - H(A_M + B_M)}_{T'_M} + A_M u + B_M v. \quad (30)$$

For DIV, let $\bar{v} = (2H - 1) - v$, which is the bitwise complement of the b -bit field v . Then

$$Q_D = \underbrace{T_D - H(A_D + B_D)}_{T'_D} + B_D + A_D u + B_D \bar{v}. \quad (31)$$

Thus operation control only selects the mode-specific table entry and either v or its short bitwise complement. Both products are unsigned and the main adder only accumulates nonnegative terms. The changes from T to T' are performed offline when the coefficient table is

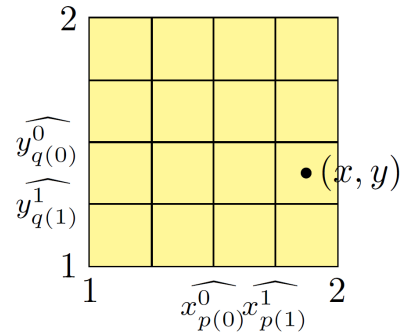


Fig. 3. Selection of the local cell and midpoint coordinates for a mantissa pair (x, y) in nested partitions.

TABLE I
Precision of the shared STDM mantissa datapath

Level	D_r	b	S	$F = S + 23 - D_r$
L0	18	5	6	11
L1	16	6	4	11
L2	16	5	5	12
L3	16	4	4	11

generated. Exhaustive checks over every retained residual code and every table entry, followed by RTL and gate-level regression, confirmed word-identical outputs to the signed forms.

Table I lists the shared-unit precisions. All mode-specific values of A , B , and T' are selected by the operation bit and the leading mantissa bits. Within each level, DIV and MUL use the same coefficient precision; in particular, the L1 shared unit uses the same $S = 4$ precision as the selected DIV-only L1 implementation.

Fig. 4 shows the resulting organization. Cell and mode bits address a compact table containing the already absorbed and recoded A , B , and T' values. The retained x field feeds one unsigned product; the retained y field is complemented only in DIV mode and feeds the other. Their outputs and T' enter a common adder, followed by fixed binary-point alignment, normalization, exponent handling, and FP32 packing. There is no level-correction chain, signed residual multiplier, wide conditional subtractor, or post-plane reciprocal multiplier.

5 Evaluation and Discussion

5.1 Evaluation Protocol

All reported PPA uses Design Compiler U-2022.12-SP7 and PrimeTime U-2022.12-SP5 with the TSMC 65-nm typical CCS library. The designs are combinational, with no clock port or sequential area. The 10-ns virtual clock has zero uncertainty, 20-ps input/output delays, an INVD0 input driver, and 0.004 output load. PT reads the synthesized netlist and SDC, checks timing, and applies static probability 0.5 and toggle rate 0.1 per 10-ns period to all inputs. PDP is the product of delay and power and is reported in fJ; ADP is the product of area and delay.

All approximate operators have 32-bit FP32 inputs and outputs and are evaluated on normal finite operands. A common wrapper extracts the two 24-bit normalized mantissas, computes the result sign and exponent, applies the normalization shift reported by the mantissa core, and packs the FP32 output. The wrapper is included in every reported PPA result. Subnormal numbers, infinities, NaNs, and exponent overflow or underflow are outside the present evaluation scope.

The DIV-only evaluation includes the four H-recoded STDM specializations, author-provided PACE mantissa RTL, and local reconstructions of PLSAD and FPD2D. PACE is connected to its normal-FP32 wrapper without

changing its mantissa arithmetic. PLSAD reconstructs the paper's ten-bit, eight-plane configuration with an eight-bit exact upper adder. FPD2D reconstructs its 8×8 fused linear architecture; because the printed coefficient table contains anomalous cells, the three reported rows use an independent implementation of the paper's coefficient-optimization algorithm rather than silently modifying the printed table. These reconstructions are not presented as author RTL. STDM, PACE, PLSAD, and FPD2D are evaluated on the same 200,000 positive normalized FP32 operand pairs and complete packed outputs. FaNZed, TruncApp, LEAD, and QIAD remain as broader local implementation context; their arithmetic and previously documented accuracy sets are unchanged. LEAD's supplied two phases are combinationaly unrolled and verified against the original two-phase RTL.

The complete DIV+MUL evaluation uses one external reference. The exact reference instantiates DesignWare DW_fp_div and DW_fp_mult in parallel and selects their outputs with the same operation control exposed by STDM. Both DesignWare operators are combinational FP32 blocks and are included in full, so this reference contains no arithmetic sharing between division and multiplication. It is a cost reference for exact FP32 support, not an accuracy-matched competitor. No prior approximate combined unit is placed in the table: SIMDive has an integer SIMD FPGA interface, while the other fused units reviewed in Section II compute exact results with different timing contracts.

5.2 Theoretical Error Analysis

Let $f(x, y) = x/y$ and let (k_x^n, k_y^n) be the midpoint of the selected level- n cell. The first-order Taylor terms give Eq. (23). Since $f_{xx} = 0$, $f_{xy} = -1/y^2$, and $f_{yy} = 2x/y^3$, the remainder can be written as

$$e_n = \frac{x}{y} - \hat{d}_n = \frac{r_y^n(r_y^n \xi_x - r_x^n \xi_y)}{\xi_y^3}, \quad (32)$$

where $\xi_x = k_x^n + \theta r_x^n$, $\xi_y = k_y^n + \theta r_y^n$, and $0 < \theta < 1$. A level- n cell has width 2^{-n} , so $|r_x^n|, |r_y^n| \leq 2^{-(n+1)}$. With normalized mantissas,

$$|e_n| \leq |r_y^n|(|2|r_y^n| + 2|r_x^n|) \leq 4^{-n}. \quad (33)$$

Thus partition refinement reduces the bound on the tangent approximation by a factor of four per level.

Residual truncation introduces a separate error. Write $\tilde{r}_x^n = r_x^n - \epsilon_x$ and $\tilde{r}_y^n = r_y^n - \epsilon_y$, where $0 \leq \epsilon_x, \epsilon_y < \Delta_r = 2^{D_r-23}$. Before coefficient quantization, the divider error becomes

$$e_n^{(r)} = e_n + c_n(k_y^n \epsilon_x - k_x^n \epsilon_y). \quad (34)$$

The opposite signs allow partial cancellation. A simple cell-dependent bound is

$$|e_n^{(r)} - e_n| < c_n(k_x^n + k_y^n)\Delta_r. \quad (35)$$

Shared STDM mantissa datapath after coefficient absorption and H recoding

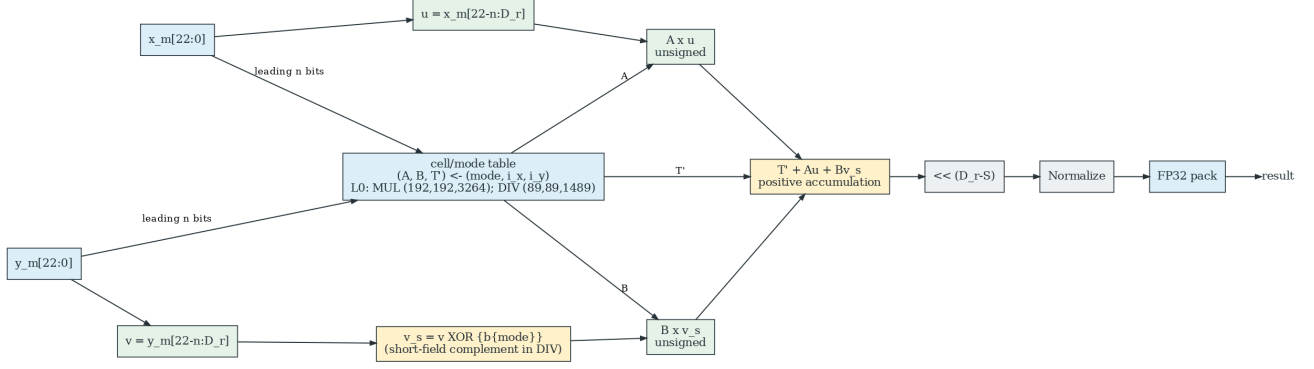


Fig. 4. Circuit structure of the shared STDM FP32 divider-multiplier after coefficient absorption and exact unsigned residual recoding.

TABLE II
DIV-only accuracy and hardware results. STDM, PACE, PLSAD, and FPD2D use the same 200,000-input accuracy set.

Design	MAE	MRED	RMSE	Area (μm^2)	Delay (ns)	Power (mW)	PDP (fJ)	Norm. ADP
Exact DesignWare DIV	–	–	–	19604.52	9.750	1.89509	18477.43	1.0000
STDM L0	0.042818	0.042856	0.056227	525.60	1.401	0.02103	29.47	0.0039
STDM L1	0.011796	0.011263	0.016233	809.64	1.685	0.03132	52.78	0.0071
STDM L2	0.003778	0.003568	0.005105	1030.32	1.940	0.03094	60.03	0.0105
STDM L3	0.002663	0.002577	0.003430	1149.12	2.006	0.03004	60.26	0.0121
PACE L1	0.028163	0.025951	0.045176	990.00	2.214	0.04467	98.92	0.0115
PACE L2	0.015143	0.014859	0.022305	1551.60	2.362	0.07829	184.88	0.0192
PACE L3	0.005897	0.005608	0.009281	2271.96	2.821	0.13119	370.08	0.0335
PACE L4	0.003856	0.003816	0.006060	3150.72	2.987	0.18531	553.51	0.0492
PLSAD ($m = 8$)	0.008975	0.008709	0.013227	659.88	1.738	0.02414	41.96	0.0060
FPD2D $8 \times 8, t = 17$	0.005344	0.005282	0.006828	790.56	1.873	0.01981	37.09	0.0077
FPD2D $8 \times 8, t = 16$	0.004054	0.004096	0.005134	869.76	1.962	0.02328	45.67	0.0089
FPD2D $8 \times 8, t = 15$	0.003653	0.003735	0.004636	942.12	2.038	0.02660	54.22	0.0100
FaNZeD	0.031811	0.031130	0.046191	2775.96	4.968	0.08880	441.16	0.0721
TruncApp	0.120105	0.119617	0.162391	4581.36	3.133	0.26136	818.77	0.0751
LEAD	0.021750	0.020993	0.028841	6666.12	5.674	0.60630	3440.38	0.1979
QIAD	0.005889	0.005880	0.006940	12097.80	8.074	0.95201	7686.66	0.5110

Finally, let A_D^* , B_D^* , and T_D^* denote the ideally scaled coefficients in Eq. (27), and define their integer quantization errors as $\delta_A = A_D - A_D^*$, $\delta_B = B_D - B_D^*$, and $\delta_T = T_D - T_D^*$. Coefficient absorption adds

$$e_n^{(q)} - e_n^{(r)} = 2^{-F}(\delta_T + \delta_A \rho_x - \delta_B \rho_y). \quad (36)$$

This term depends on both the cell and retained residuals, so choosing a base reciprocal solely by its distance from $1/(k_y^n)^2$ need not minimize the error of the implemented

FP32 divider. The unsigned recoding in Eqs. (30) and (31) is an integer identity and adds no error.

5.3 Reciprocal Coefficient Calibration

For each y interval, neighboring fixed-point reciprocal codes are evaluated through residual truncation, absorbed coefficient generation, normalization, and FP32 packing. If $C_0 = 1/(k_y^n)^2$, w denotes the corresponding truncated plane, and $e_0 = x/y - C_0 w$, replacing C_0 by $C_0 + \delta$ changes

TABLE III
Accuracy and hardware results for designs supporting both DIV and MUL.

Design	DIV MRED	MUL MRED	Area (μm^2)	Delay (ns)	Power (mW)	PDP (fJ)	ADP ($\mu\text{m}^2\text{ns}$)
Exact DesignWare DIV+MUL	–	–	24025.68	9.856	2.08421	20541.66	236793.50
STDM L0	0.042856	0.032453	891.36	1.836	0.03415	62.71	1636.94
STDM L1	0.011263	0.007903	1183.32	2.136	0.03932	83.99	2527.95
STDM L2	0.003568	0.002687	1454.40	2.401	0.03797	91.17	3492.15
STDM L3	0.002577	0.001901	1794.96	2.509	0.03944	98.95	4503.29

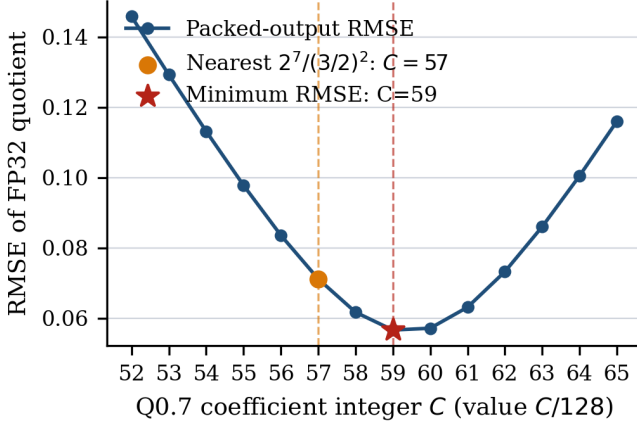


Fig. 5. L0 end-to-end reciprocal-code calibration. The code nearest to $4/9$ is 57, whereas code 59 minimizes the complete-divider RMSE before coefficient absorption.

the mean squared error according to

$$E[e^2] = E[e_0^2] - 2\delta E[e_0 w] + \delta^2 E[w^2]. \quad (37)$$

Because plane error and coefficient error are correlated, the nearest code is not necessarily the best end-to-end code. For example, the ideal L0 value is $4/9$ and its nearest Q0.7 code is $57/128$, but the complete calibration sweep selects $59/128$ for minimum RMSE. This code is subsequently folded into A_D , B_D , and T_D ; 59 is not a standalone multiplier operand in the final circuit.

5.4 DIV-Only Comparison

Table II compares the four STDM dividers with PACE and the locally implemented prior dividers. All hardware results use the common FP32 interface and synthesis conditions described in Section 5.1. The ADP values are normalized to the exact DesignWare divider. These rows are DIV-only specializations, whereas Table III reports the complete shared units.

STDM L1–L3 have lower MAE, MRED, and RMSE than PACE L2–L4, respectively, while using less area, delay, and power in every paired comparison. PLSAD is smaller than STDM L1 but lies between STDM L1 and L2 in all three error measures. The reconstructed FPD2D points define a strong low-cost region: $8 \times 8, t = 16$ nearly matches STDM L2, while $t = 15$ trades lower RMSE for higher MRED. STDM L3 is more accurate than all three evaluated FPD2D points, but requires more area and power than $t = 15$. Thus STDM does not claim the best isolated divider at every error target; its additional value is a divider plane that can share its dominant arithmetic with multiplication. STDM L3 uses 94.1% less area, 79.4% less delay, and 98.4% less power than the exact divider, with a normalized ADP of 0.0121.

TABLE IV

Effect of sharing arithmetic between DIV and MUL. Percentages in Shared rows are changes relative to Separate at the same level.

Level	Structure	Area (μm^2)	Delay (ns)	Power (mW)	PDP (fJ)	ADP ($\mu\text{m}^2\text{ns}$)
L0	Separate	911.88	1.649	0.03451	56.89	1503.44
	Shared	891.36 (-2.3%)	1.836 (+11.3%)	0.03415 (-1.0%)	62.71 (+10.2%)	1636.94 (+8.9%)
L1	Separate	1429.20	2.005	0.05746	115.23	2866.09
	Shared	1183.32 (-17.2%)	2.136 (+6.5%)	0.03932 (-31.6%)	83.99 (-27.1%)	2527.95 (-11.8%)
L2	Separate	1796.40	2.271	0.06416	145.71	4079.48
	Shared	1454.40 (-19.0%)	2.401 (+5.7%)	0.03797 (-40.8%)	91.17 (-37.4%)	3492.15 (-14.4%)
L3	Separate	2196.72	2.374	0.06850	162.62	5215.28
	Shared	1794.96 (-18.3%)	2.509 (+5.7%)	0.03944 (-42.4%)	98.95 (-39.2%)	4503.29 (-13.7%)

5.5 DIV and MUL Comparison

Table III reports complete STDM units and the exact DesignWare cost reference. The exact design contains separate DIV and MUL operators followed by an output selector, so it does not isolate the benefit of sharing and is not accuracy matched to STDM. Compared with the exact DIV+MUL reference, STDM L3 reduces area by 92.5%, delay by 74.5%, power by 98.1%, and ADP by 98.1%.

To measure the benefit of sharing itself, Table IV compares two implementations with identical arithmetic results. In the Separate implementation, DIV and MUL use independent mantissa cores. A mode selector chooses one of the two core outputs, after which normalization and FP32 packing are shared. In the Shared implementation, the mantissa datapath also shares cell decoding, both unsigned residual products, the main adder, and the output path. Mode control selects a coefficient-table entry and complements the short y residual for division. Both structures use coefficient absorption and H recoding, and they produce identical outputs for both operations at every level.

From L1 to L3, sharing reduces area by 17.2–19.0%, power by 31.6–42.4%, and ADP by 11.8–14.4%, while increasing delay by at most 6.5%. L0 saves 2.3% area and 1.0% power, but its delay increases by 11.4%, causing an 8.9% ADP increase. At L0 the operation control enters the critical path; at the higher levels, the partition and mantissa arithmetic dominate the delay, so the selection overhead is much less pronounced. These results show that the Shared implementation provides a clear overall benefit from L1 onward, while the Separate implementation is preferable at L0 when ADP is the primary objective.

6 Conclusion

STDM uses the algebraic relation between divider and multiplier tangent planes to implement both FP32 operations with one affine mantissa datapath. Absorbing the reciprocal scale into cell coefficients removes the post-plane multiplier, while exact H recoding converts the retained centered residuals into two unsigned products and a positive accumulation. The transform changes hardware organization without changing the approximate outputs. DIV-only results show that STDM occupies a

distinct accuracy–cost region alongside PACE, PLSAD, and FPD2D rather than dominating every specialized divider. Its main advantage appears when both operations are required: from L1 to L3, sharing saves 17.2–19.0% area and 31.6–42.4% power over matched separate cores. At L3, the complete unit reduces area, delay, power, and ADP by 92.5%, 74.5%, 98.1%, and 98.1%, respectively, relative to the exact DIV+MUL reference.

References

- [1] C. Chen, S. Yang, W. Qian, M. Imani, X. Yin, and C. Zhuo, "Optimally approximated and unbiased floating-point multiplier with runtime configurability," in *Proc. IEEE/ACM ICCAD*, 2020, pp. 1–9.
- [2] C. Wen, H. Du, J. Wang, Z. Chen, L. Zhang, Q. Sun, and C. Zhuo, "PACE: A piece-wise approximate floating-point divider with runtime configurability and high energy efficiency," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 2, Art. 21, 2025, doi: 10.1145/3706634.
- [3] Z. Ebrahimi, S. Ullah, and A. Kumar, "SIMDive: Approximate SIMD soft multiplier-divider for FPGAs with tunable accuracy," in *Proc. GLSVLSI*, 2020.
- [4] C. Chen, H.-L. Yen, and M.-H. Sheu, "Design of a cost-effective multiplier/divider fused unit for floating-point arithmetic," *Int. J. Electr. Eng.*, vol. 11, no. 3, pp. 239–246, 2004.
- [5] T.-J. Kwon, J.-S. Moon, J. Sondeen, and J. Draper, "A 0.18- μ m implementation of a floating-point unit for a processing-in-memory system," in *Proc. IEEE ISCAS*, 2004, vol. 2, pp. II-453–II-456, doi: 10.1109/ISCAS.2004.1329306.
- [6] Frontgrade Gaisler, "GRFPU: High-performance IEEE-754 floating-point unit," [Online]. Available: <https://www.gaisler.com/products/ieee-754-floating-point-unit>.
- [7] L. Chen, J. Han, W. Liu, and F. Lombardi, "Design of approximate unsigned integer non-restoring divider for inexact computing," in *Proc. GLSVLSI*, 2015, pp. 51–56, doi: 10.1145/2742060.2742063.
- [8] S. Hashemi, R. I. Bahar, and S. Reda, "A low-power dynamic divider for approximate applications," in *Proc. DAC*, 2016, pp. 1–6, doi: 10.1145/2897937.2897965.
- [9] M. Imani, R. Garcia, A. Huang, and T. Rosing, "CADE: Configurable approximate divider for energy efficiency," in *Proc. DATE*, 2019, pp. 586–589, doi: 10.23919/DATE.2019.8715112.
- [10] J. Melchert, S. Behroozi, J. Li, and Y. Kim, "SAADI-EC: A quality-configurable approximate divider for energy efficiency," *IEEE Trans. VLSI Syst.*, vol. 27, no. 11, pp. 2680–2692, 2019, doi: 10.1109/TVLSI.2019.2926083.
- [11] J. Oelund and S. Kim, "ILAFD: Accuracy-configurable floating-point divider using an approximate reciprocal and an iterative logarithmic multiplier," in *Proc. GLSVLSI*, 2023, pp. 639–644, doi: 10.1145/3583781.3590262.
- [12] H. Saadat, H. Javaid, and S. Parameswaran, "Approximate integer and floating-point dividers with near-zero error bias," in *Proc. DAC*, 2019, doi: 10.1145/3316781.3317773.
- [13] S. Vahdat, M. Kamal, A. Afzali-Kusha, M. Pedram, and Z. Navabi, "TruncApp: A truncation-based approximate divider for energy efficient DSP applications," in *Proc. DATE*, 2017, doi: 10.23919/DATE.2017.7927254.
- [14] O. G. Ratnaparkhi and M. Rao, "LEAD: Logarithmic exponent approximate divider for image quantization application," in *Proc. GLSVLSI*, 2022, doi: 10.1145/3526241.3530323.
- [15] H. Liu, M.-J. Wang, and M. Liu, "QIAD: A quadratic interpolation approximate divider," *IEICE Electronics Express*, vol. 20, no. 11, 2023, doi: 10.1587/elex.20.20230167.
- [16] L. Wu, "A curve fitting approach for non-iterative divider design with accuracy and performance trade-off," in *Proc. NEWCAS*, 2015, doi: 10.1109/NEWCAS.2015.7182097.
- [17] C. Wu, W. Shi, Y. Yuan, Z. Zou, Z. Mo, and J. He, "Area-delay-energy-efficient approximate dividers based on piece-wise linear fitting of surface," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 11, pp. 5017–5029, 2024, doi: 10.1109/TCSI.2024.3425538.
- [18] G. Di Meo, A. G. M. Strollo, D. De Caro, L. Tegazzini, and E. Napoli, "Low-power high precision floating-point divider with bidimensional linear approximation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 882–895, 2025, doi: 10.1109/TCSI.2024.3447830.
- [19] IEEE Standard for Floating-Point Arithmetic, IEEE Standard 754-2008, pp. 1–70, 2008.